

Chapter 25

Multivariable Calculus for Data Science and Machine Learning

Chapter overview

Machine learning training is an optimization problem on a high-dimensional parameter space. A model turns inputs into predictions, a loss function turns prediction errors into a single number, and calculus tells us how that number changes as the parameters move. The main objects are familiar: gradients, Jacobians, Hessians, chain rules, tangent approximations, and level sets. What changes in modern data science is scale: instead of two or three variables, the parameter vector may contain thousands, millions, or billions of components.

Learning goals

By the end of this chapter, you should be able to:

1. translate a learning problem into a multivariable function of parameters;
2. identify loss functions, empirical risk, gradients, Jacobians, and Hessians;
3. derive gradients for linear regression, logistic regression, and simple neural-network models;
4. interpret gradient descent as motion on a high-dimensional surface;
5. explain the role of regularization in optimization and statistical learning;
6. use computational visualization to study loss landscapes and gradient-based training;
7. connect calculus ideas to automatic differentiation, backpropagation, and modern AI workflows.

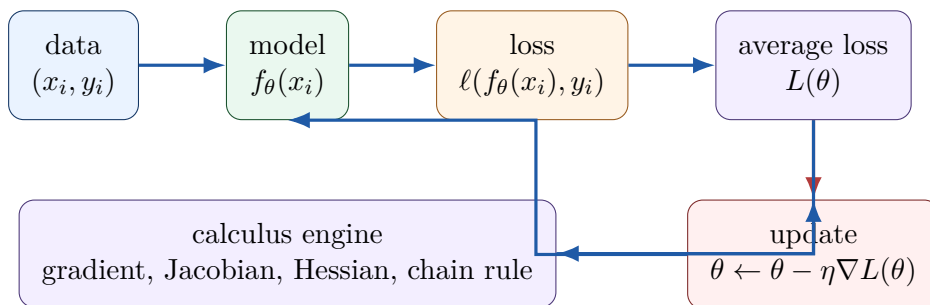


Figure 25.1: A supervised learning workflow viewed as multivariable calculus. The parameter vector θ is adjusted by information from derivatives of the loss.

25.1 Why machine learning is multivariable calculus

A supervised learning problem begins with data

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n),$$

where each input x_i is often a vector in $\mathbb{R}^d$ and each output y_i may be a number, a class label, or another vector. A model is a family of functions

$$f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^m,$$

where $\theta \in \mathbb{R}^p$ is a vector of parameters. Training the model means choosing θ so that $f_\theta(x_i)$ is close to y_i for the observed data and hopefully remains accurate on new data.

The central object is a *loss function*

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_\theta(x_i), y_i),$$

a function from parameter space to the real line:

$$L : \mathbb{R}^p \rightarrow \mathbb{R}.$$

So training is not mysterious from the viewpoint of calculus: it is optimization for a function of many variables.

Main principle

A model turns inputs into predictions. A loss function turns predictions into a number measuring error. Optimization changes the parameters to reduce that number.

The phrase *high-dimensional calculus* is not just a metaphor. In a small linear regression model, the parameter vector may have a few components. In a modern neural network, it may have many more. The mathematical objects remain the same.

Calculus object	Machine learning role
Point $\theta \in \mathbb{R}^p$	Current parameter values
Function $L(\theta)$	Loss or objective
Gradient $\nabla L(\theta)$	Direction of steepest increase
Negative gradient $-\nabla L(\theta)$	Direction of steepest local decrease
Hessian $H_L(\theta)$	Local curvature of the loss
Jacobian $J_f(\theta)$	Sensitivity of a vector-valued model
Chain rule	Backpropagation and automatic differentiation

25.2 A first loss function: least squares

Suppose the data are approximately linear:

$$y_i \approx \beta_0 + \beta_1 x_i.$$

The model has two parameters $\theta = (\beta_0, \beta_1)$. For a single data point, the prediction error is

$$e_i(\beta_0, \beta_1) = y_i - (\beta_0 + \beta_1 x_i).$$

The least-squares loss is

$$L(\beta_0, \beta_1) = \frac{1}{n} \sum_{i=1}^n (y_i - (\beta_0 + \beta_1 x_i))^2.$$

This is a surface over the (β_0, β_1) -plane. Training means finding the lowest point on this surface.

Worked example: gradient of least-squares loss

For

$$L(\beta_0, \beta_1) = \frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2,$$

let

$$r_i = y_i - \beta_0 - \beta_1 x_i.$$

Then

$$L = \frac{1}{n} \sum_{i=1}^n r_i^2.$$

Differentiate with respect to β_0 :

$$\frac{\partial L}{\partial \beta_0} = \frac{1}{n} \sum_{i=1}^n 2r_i \frac{\partial r_i}{\partial \beta_0} = -\frac{2}{n} \sum_{i=1}^n r_i.$$

Differentiate with respect to β_1 :

$$\frac{\partial L}{\partial \beta_1} = \frac{1}{n} \sum_{i=1}^n 2r_i \frac{\partial r_i}{\partial \beta_1} = -\frac{2}{n} \sum_{i=1}^n x_i r_i.$$

Therefore

$$\nabla L(\beta_0, \beta_1) = \left\langle -\frac{2}{n} \sum_{i=1}^n r_i, -\frac{2}{n} \sum_{i=1}^n x_i r_i \right\rangle.$$

At a critical point the residuals satisfy

$$\sum_{i=1}^n r_i = 0, \quad \sum_{i=1}^n x_i r_i = 0.$$

Geometrically, the residual vector is orthogonal to the columns of the design matrix.

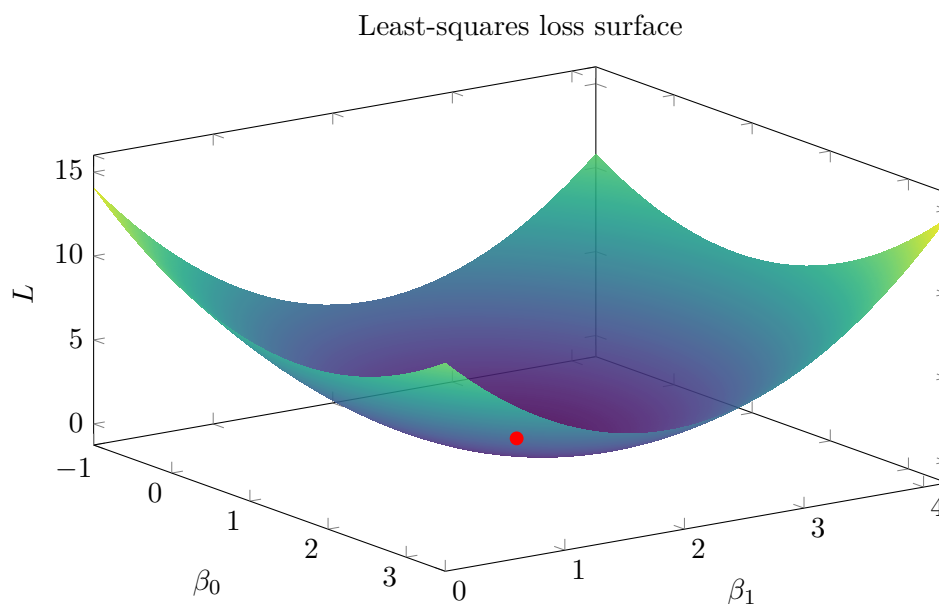


Figure 25.2: A least-squares loss surface. Training searches for the lowest point in parameter space.

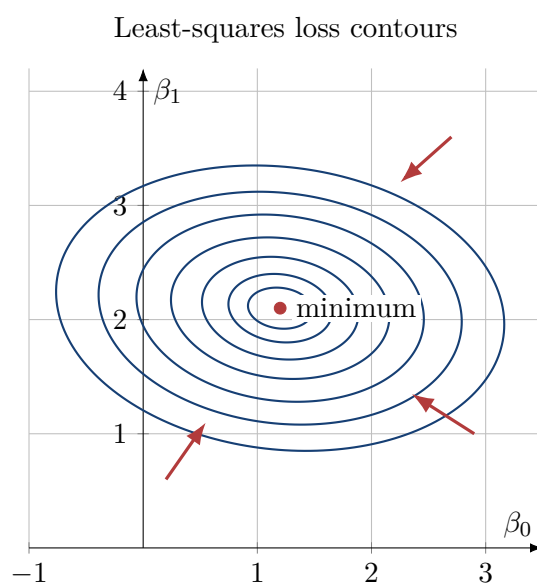


Figure 25.3: Contour lines of the least-squares loss. The gradient is perpendicular to the contours; the negative gradient points downhill.

25.3 Matrix form and the geometry of linear models

For multiple features, place the data in a design matrix

$$X = \begin{bmatrix} - & x_1^\top & - \\ - & x_2^\top & - \\ & \vdots & \\ - & x_n^\top & - \end{bmatrix}, \quad w \in \mathbb{R}^d, \quad \hat{y} = Xw.$$

The mean squared error loss is

$$L(w) = \frac{1}{n} \|Xw - y\|^2.$$

The gradient is

$$\nabla L(w) = \frac{2}{n} X^\top (Xw - y).$$

This formula is a bridge between multivariable calculus and linear algebra. The residual vector $Xw - y$ lives in data space $\mathbb{R}^n$, while the gradient lives in parameter space $\mathbb{R}^d$.

At a critical point,

$$X^\top (Xw - y) = 0.$$

Thus the residual vector is orthogonal to every column of X . This is the projection geometry behind least squares.

Geometry of least squares

The least-squares solution chooses Xw to be the orthogonal projection of y onto the column space of X . The calculus condition $\nabla L(w) = 0$ is the same as the linear algebra condition that the residual is orthogonal to the model space.

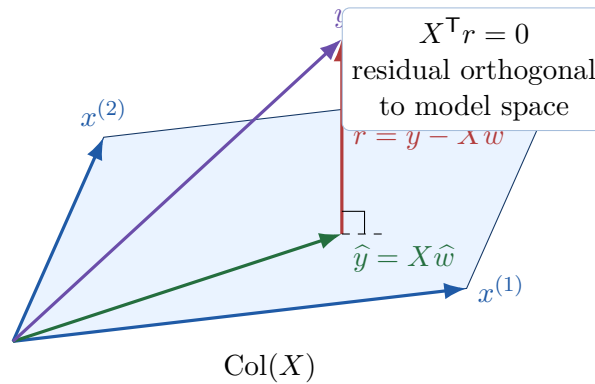


Figure 25.4: Projection geometry of least squares. The best prediction $X\hat{w}$ is the projection of y onto the column space of X .

Gradient descent for linear regression

Gradient descent updates parameters by

$$w_{k+1} = w_k - \eta \nabla L(w_k),$$

where $\eta > 0$ is the learning rate. For mean squared error,

$$w_{k+1} = w_k - \eta \frac{2}{n} X^T (Xw_k - y).$$

If the learning rate is too small, training is slow. If it is too large, the method may oscillate or diverge.

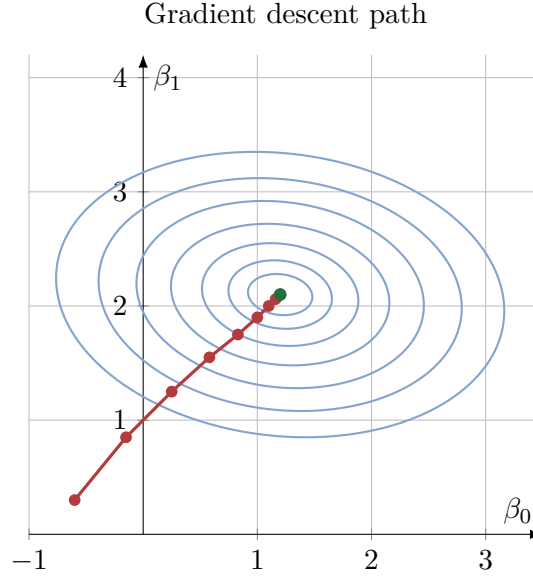


Figure 25.5: Gradient descent follows the negative gradient direction across loss contours.

25.4 Empirical risk and the structure of training problems

Most supervised learning problems can be written as empirical risk minimization:

$$\min_{\theta} L(\theta) = \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i).$$

There are three layers:

1. **Model layer:** $f_{\theta}(x)$ converts an input into a prediction.
2. **Loss layer:** $\ell(f_{\theta}(x_i), y_i)$ measures the error on one data point.
3. **Optimization layer:** the average loss is minimized over θ .

For a scalar-output model, the chain rule gives the gradient of one training example:

$$\nabla_{\theta} \ell(f_{\theta}(x_i), y_i) = \frac{\partial \ell}{\partial \hat{y}_i} \nabla_{\theta} f_{\theta}(x_i), \quad \hat{y}_i = f_{\theta}(x_i).$$

For a vector-output model $f_{\theta}(x_i) \in \mathbb{R}^m$, the Jacobian appears:

$$\nabla_{\theta} \ell(f_{\theta}(x_i), y_i) = J_{f_{\theta}}(x_i)^T \nabla_{\hat{y}_i} \ell(\hat{y}_i, y_i).$$

This is the central calculus identity behind backpropagation.

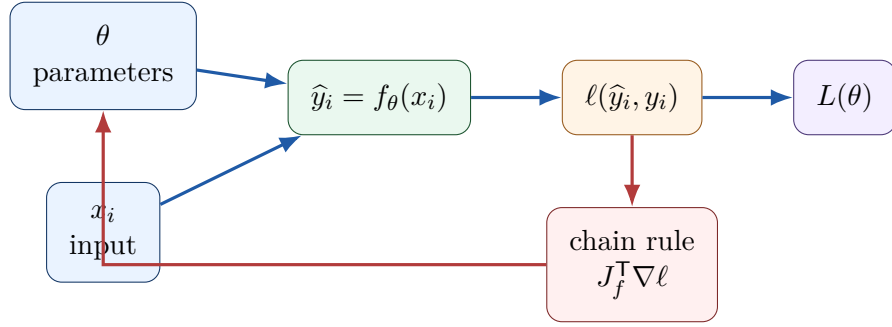


Figure 25.6: Empirical risk has a model layer, a loss layer, and an optimization layer. The chain rule transports derivatives backward to the parameters.

25.5 Logistic regression and classification

For binary classification, labels are often $y_i \in \{0, 1\}$. Logistic regression uses

$$p_i = \sigma(w^\top x_i + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The output p_i is interpreted as a probability that $y_i = 1$. The binary cross-entropy loss is

$$L(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Let $z_i = w^\top x_i + b$ and $p_i = \sigma(z_i)$. A useful derivative is

$$\frac{d}{dz} \sigma(z) = \sigma(z)(1 - \sigma(z)).$$

For the cross-entropy loss, the derivative simplifies:

$$\frac{\partial L}{\partial z_i} = \frac{1}{n} (p_i - y_i).$$

Therefore

$$\nabla_w L(w, b) = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) x_i, \quad \frac{\partial L}{\partial b} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i).$$

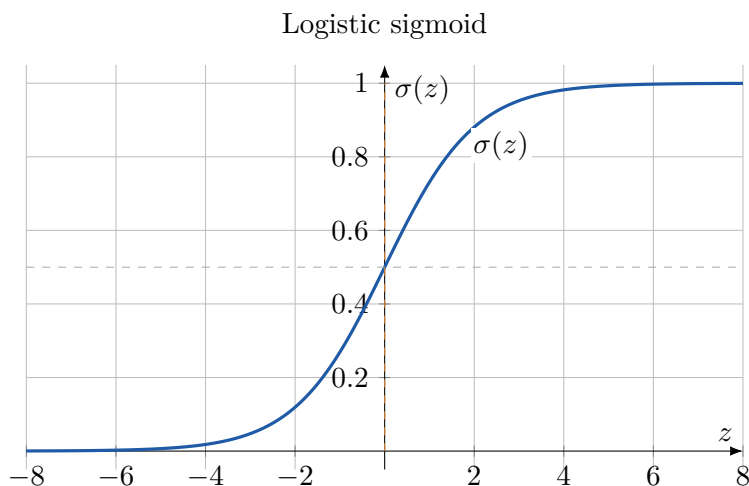


Figure 25.7: The logistic sigmoid transforms real-valued scores into probabilities between 0 and 1.

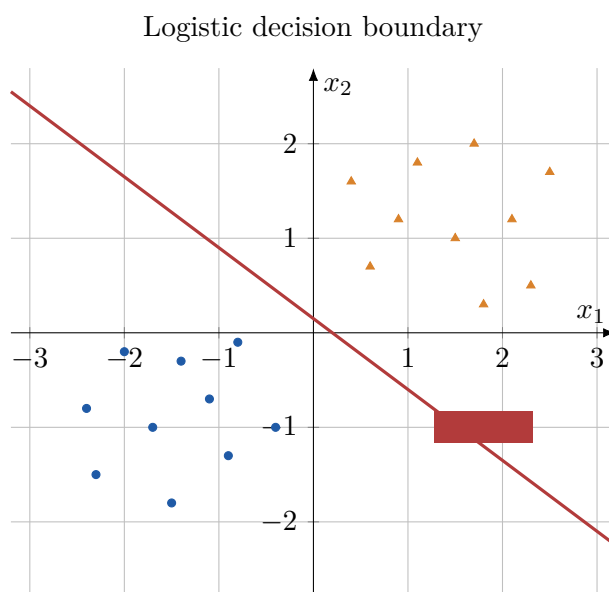


Figure 25.8: A logistic regression model learns a linear decision boundary in feature space.

25.6 Regularization and the bias-variance tradeoff

A loss function measures fit to data. A regularization term measures model complexity. A common objective is

$$J(\theta) = L(\theta) + \lambda R(\theta),$$

where $\lambda \geq 0$ controls the strength of regularization.

For ridge regression,

$$R(w) = \|w\|^2.$$

The regularized objective is

$$J(w) = \frac{1}{n} \|Xw - y\|^2 + \lambda \|w\|^2.$$

Its gradient is

$$\nabla J(w) = \frac{2}{n} X^\top (Xw - y) + 2\lambda w.$$

The term $2\lambda w$ pulls the parameter vector toward the origin. This often improves generalization by preventing unstable parameter estimates.

Calculus interpretation of regularization

Regularization changes the geometry of the loss surface. It adds curvature that penalizes large parameter values, often making the optimization problem better conditioned.

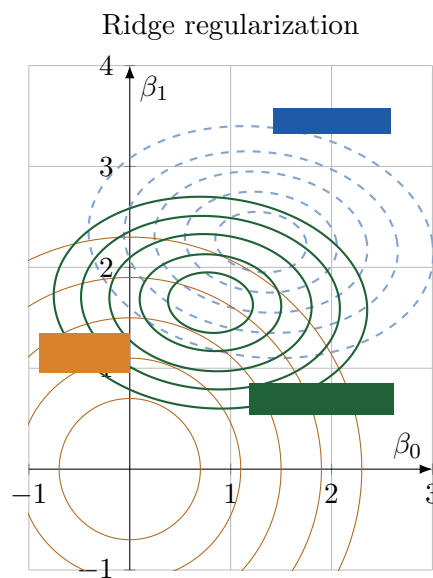


Figure 25.9: Ridge regularization adds circular penalty contours to the data-fitting loss, shifting the optimum toward smaller coefficients.

25.7 Stochastic gradient descent

For large data sets, computing the full gradient

$$\nabla L(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} \ell_i(\theta)$$

may be expensive. Stochastic gradient descent replaces the full average by a random estimate:

$$\theta_{k+1} = \theta_k - \eta_k g_k,$$

where g_k is computed from one data point or a mini-batch. A mini-batch gradient has the form

$$g_k = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla_{\theta} \ell_i(\theta_k).$$

This introduces noise into the path. The noise can slow convergence near a minimum, but it can also help training move through complicated landscapes.

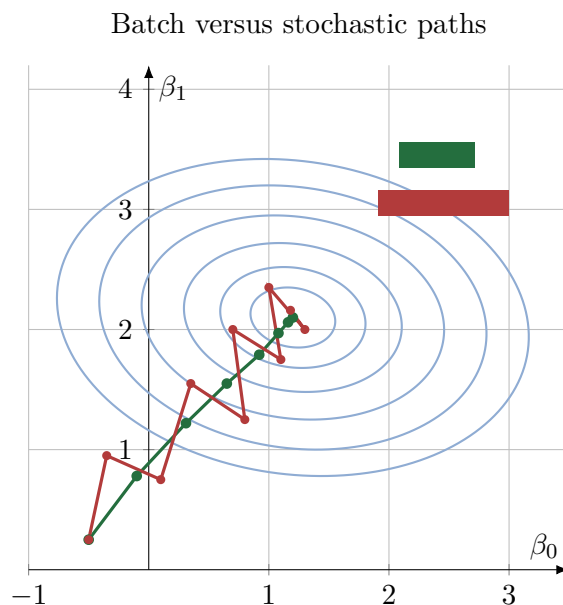


Figure 25.10: Batch gradient descent follows a smooth path; stochastic gradient descent follows a noisier path.

25.8 Neural networks and the chain rule

A neural network is a composition of functions. A simple one-hidden-layer network has the form

$$f_{\theta}(x) = W_2 \phi(W_1 x + b_1) + b_2,$$

where ϕ is an activation function applied componentwise. The parameters are

$$\theta = (W_1, b_1, W_2, b_2).$$

The main calculus idea is not new: the derivative of a composition is computed by the chain rule. What changes is scale. A network may have many layers:

$$x \mapsto h_1 \mapsto h_2 \mapsto \cdots \mapsto h_K \mapsto \hat{y} \mapsto L.$$

Backpropagation computes derivatives from the output backward through the computational graph.

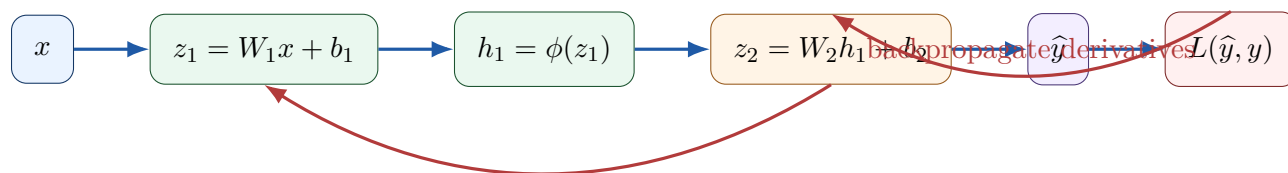


Figure 25.11: A neural network is a composition. Backpropagation is the chain rule applied backward through the computational graph.

A scalar example

Let

$$\hat{y} = a \phi(wx + b), \quad L = \frac{1}{2}(\hat{y} - y)^2.$$

Let $z = wx + b$ and $h = \phi(z)$. Then $\hat{y} = ah$. By the chain rule,

$$\frac{\partial L}{\partial a} = (\hat{y} - y)h,$$

$$\frac{\partial L}{\partial w} = (\hat{y} - y)a\phi'(z)x,$$

and

$$\frac{\partial L}{\partial b} = (\hat{y} - y)a\phi'(z).$$

Every large backpropagation computation is built from this same pattern.

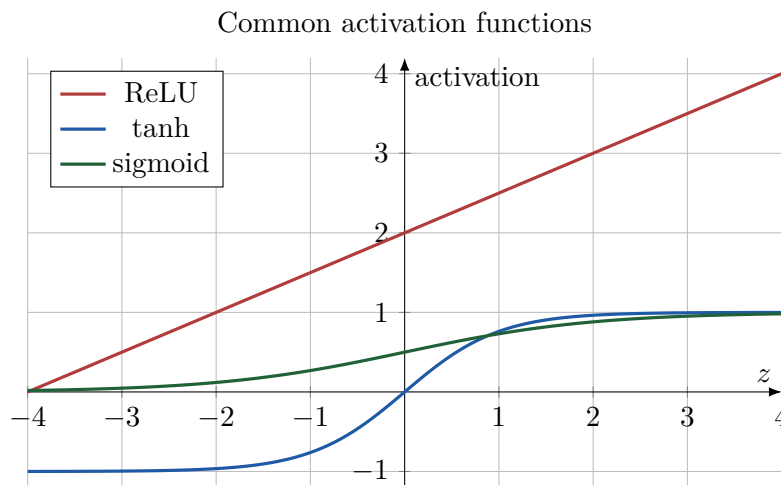


Figure 25.12: Common activation functions shape how neural networks bend space.

25.9 Numerical gradient checking

Even when gradients are computed automatically, it is useful to understand numerical checks. For a differentiable function $L : \mathbb{R}^p \rightarrow \mathbb{R}$, the central difference approximation to the j th partial derivative is

$$\frac{\partial L}{\partial \theta_j}(\theta) \approx \frac{L(\theta + he_j) - L(\theta - he_j)}{2h}.$$

This checks whether an analytic or automatic gradient is plausible.

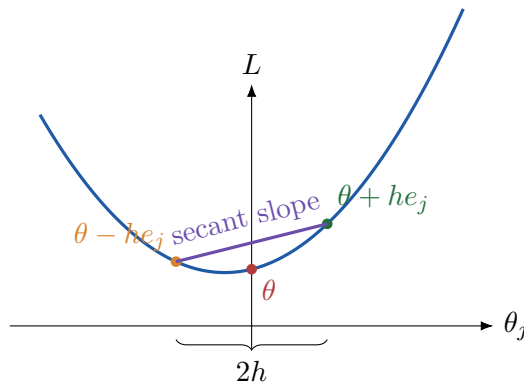


Figure 25.13: A central difference uses two nearby function values to approximate one component of the gradient.

A small difference between the analytic and numerical gradients gives confidence that the derivative was implemented correctly. A large difference suggests an algebra error, a coding error, or a nondifferentiability issue.

25.10 Hessians, curvature, and learning rates

Near a point θ_0 , a twice differentiable loss has the second-order approximation

$$L(\theta_0 + u) \approx L(\theta_0) + \nabla L(\theta_0)^\top u + \frac{1}{2} u^\top H_L(\theta_0) u.$$

The Hessian tells us how curved the loss is in different directions. If the Hessian has eigenvalues with very different sizes, the level sets are long and narrow. Gradient descent may zigzag.

For least squares,

$$L(w) = \frac{1}{n} \|Xw - y\|^2,$$

the Hessian is constant:

$$H_L(w) = \frac{2}{n} X^\top X.$$

The eigenvalues of $X^\top X$ control the conditioning of the optimization problem.



Figure 25.14: Poor conditioning creates narrow valleys, making gradient descent sensitive to learning rate and prone to zigzagging.

25.11 Feature maps and nonlinear models

A model can be nonlinear in the input but linear in parameters. For example,

$$f_w(x) = w_0 + w_1x + w_2x^2.$$

This is linear regression after replacing x by the feature vector

$$\phi(x) = (1, x, x^2).$$

More generally, a feature map

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^m$$

transforms the original input into a space where a simpler model may work better.

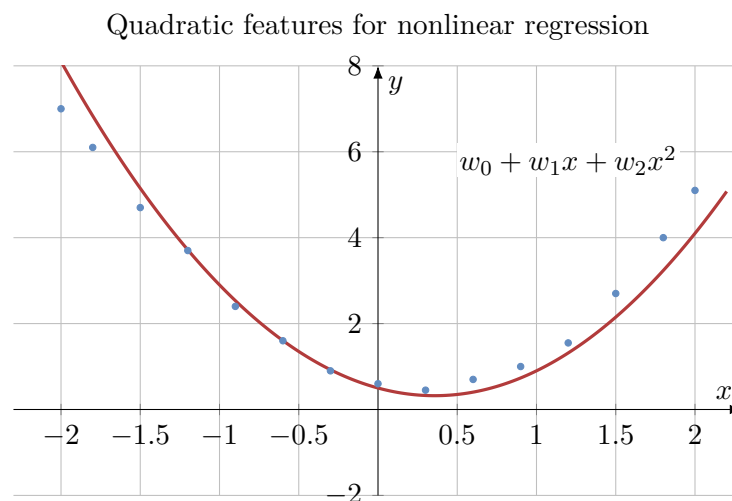


Figure 25.15: A nonlinear feature map can turn a curved pattern into a linear problem in transformed coordinates.

This example separates two ideas:

- *nonlinear in the input* means the graph may bend;
- *linear in the parameters* means the loss may still be a convex quadratic.

Neural networks are more complicated because they are usually nonlinear in both inputs and parameters.

25.12 What calculus tells us about machine learning

Multivariable calculus gives a precise language for several major questions in machine learning.

Sensitivity

How much does the output change when the input or parameters change? This is measured by derivatives and Jacobians.

Training

How should parameters be changed to reduce the loss? This is guided by gradients and optimization algorithms.

Curvature

Why is one learning problem easy and another hard? Hessians, eigenvalues, and conditioning provide part of the answer.

Generalization

Why might a model fit training data but fail on new data? Regularization, model complexity, and stability are connected to the geometry of the parameter space.

Computation

How can derivatives be computed for huge composed functions? The answer is the chain rule organized as automatic differentiation.

The big picture

Machine learning does not replace calculus. It scales calculus. The same gradient, Jacobian, Hessian, and chain-rule ideas from this book become computational tools for learning from data.

25.13 Common mistakes

Common mistakes

1. **Confusing feature space and parameter space.** Inputs x live in feature space; parameters θ live in parameter space. Gradients of the loss are usually taken with respect to parameters.
2. **Using the gradient direction instead of the negative gradient direction.** The gradient points toward steepest increase. Gradient descent moves in the negative gradient direction.
3. **Choosing a learning rate without checking behavior.** A learning rate can be too small, useful, unstable, or divergent.
4. **Assuming every loss is convex.** Least squares is convex in linear parameters, but neural-network losses are usually nonconvex.
5. **Ignoring scaling.** Feature scaling changes the geometry of the loss surface and can strongly affect optimization.
6. **Trusting code without gradient checks.** Numerical derivatives are useful for debugging small models.

25.14 Practice problems

Conceptual problems

1. Explain the difference between feature space and parameter space for a linear model $f_w(x) = w^\top x$.
2. What does the loss surface represent in linear regression?
3. Why is the negative gradient direction used in gradient descent?
4. Explain why the chain rule is the mathematical foundation of backpropagation.
5. In logistic regression, why is the output interpreted as a probability?
6. What is the purpose of regularization?
7. Why can poor conditioning slow down gradient descent?
8. What is the difference between batch gradient descent and stochastic gradient descent?

Computational problems

9. For $L(w_1, w_2) = 3w_1^2 + 2w_1w_2 + w_2^2$, compute ∇L and perform three steps of gradient descent starting at $(1, 2)$ with learning rate $\eta = 0.1$.
10. Let

$$L(\beta_0, \beta_1) = \frac{1}{3} \left[(1 - \beta_0)^2 + (2 - \beta_0 - \beta_1)^2 + (4 - \beta_0 - 2\beta_1)^2 \right].$$

Compute ∇L and solve $\nabla L = 0$.

11. For logistic regression with one feature, $p_i = \sigma(wx_i + b)$, show that

$$\frac{\partial L}{\partial w} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)x_i.$$

12. For ridge regression,

$$J(w) = \frac{1}{n} \|Xw - y\|^2 + \lambda \|w\|^2,$$

compute the Hessian.

13. Let $f_w(x) = w_0 + w_1x + w_2x^2$. Write the design matrix for data points $x_1, \dots, x_n$.

14. Suppose $L(\theta) = \frac{1}{2}(f(\theta) - y)^2$. Show that

$$\nabla L(\theta) = (f(\theta) - y)\nabla f(\theta).$$

Python problems

15. Modify the gradient descent code in this chapter to use a smaller learning rate and compare the path.
16. Modify the logistic regression simulation so that the two classes overlap more. What happens to the decision boundary?
17. Implement ridge regression by gradient descent and compare the result for $\lambda = 0$, $\lambda = 0.1$, and $\lambda = 1$.
18. Write a numerical gradient checker for the logistic regression loss.
19. Simulate stochastic gradient descent with different mini-batch sizes.
20. Create a polynomial regression example with features $1, x, x^2, x^3$ and compare the fit to a linear model.

Project problems

21. **Loss landscape project.** Choose a two-parameter model and create a contour plot of its loss surface. Overlay several gradient descent paths with different learning rates.
22. **Classification project.** Generate a two-class data set, train logistic regression from scratch, and visualize the learned decision boundary.
23. **Regularization project.** Fit polynomial models of several degrees and compare training error, testing error, and coefficient size.
24. **Backpropagation project.** Implement the scalar neural-network example from Section 25.8 and compare analytic gradients with numerical gradients.

25.15 Python lab and interactive page

- Notebook lab: `Chapter25_Multivariable_Calculus_for_Data_Science_and_Machine_Learning.ipynb`
- Interactive page: `chapter-25.html`

25.16 Chapter summary

This chapter connected multivariable calculus to data science and machine learning. A model is a function with parameters, and training is the problem of minimizing a loss function on parameter space. Gradients guide local improvement, Hessians describe curvature, Jacobians measure sensitivity, and the chain rule powers backpropagation. Linear regression, logistic regression, regularization, stochastic gradient descent, and neural networks all use the same core calculus ideas developed throughout this book.